# Reviewer Pack — Minimal Root System Rank and Circuit Topology Inequality

Entry fa2dcbe02a24 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 17:54:06 UTC

## Minimal Root System Rank and Circuit Topology Inequality

**Entry ID**: `fa2dcbe02a24` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 17:54:06 UTC

### 1. Conjecture

**Field A** (mathematical branch): Lie Theory (Root Systems) **Field B** (complexity object): Boolean Circuits (Circuit Topology)

**Statement**:

> For every Boolean circuit C with n inputs, the rank of its root system R(C) is upper bounded by the number of levels in C, i.e., |R(C)| ≤ T(C), where T(C) is the depth of C. Equivalently, for any d-regular graph G representing a circuit C, the dimension of the Lie algebra associated with the root system R(G) is at most the number of vertices in G, i.e., dim(LieAlg(R(G))) ≤ |V(G)|.

**Rationale (proposer's reasoning)**:

> Root systems encode geometric information about symmetry groups, and their ranks can be used to study the structure of these groups. Circuit topology provides a way to measure the complexity of Boolean functions. By connecting the rank of a root system with the depth of a circuit, this conjecture could potentially expose a deeper connection between algebraic and computational structures.

**Taxonomy category**: `Lie Theory` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `2691ef37486c5935`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> For a Boolean circuit C with n inputs and depth T(C), if the rank of its root system R(C) is less than or equal to T(C), i.e., |R(C)| ≤ T(C), and for any d-regular graph G representing C, the dimension of the associated Lie algebra dim(LieAlg(R(G))) is less than or equal to the number of vertices in G, i.e., dim(LieAlg(R(G))) ≤ |V(G)|, then the conjecture is supported.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 1.00 | SAFE | SAFE |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 1.00 | UNCERTAIN | SAFE |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "root system rank" AND "Boolean circuit topology" - "Lie algebra dimension" AND "circuit depth" - "d-regular graph" AND "vertex count" AND Lie theory

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n, max_depth):
        if n == 1 or max_depth == 0:
            return []
        depth = random.randint(2, max_depth)
        subcircuit_size = random.randint(1, n-1)
        subcircuit = generate_random_circuit(subcircuit_size, depth - 2)
        return [subcircuit] * (n - subcircuit_size) + [[[]]]

    def adjacency_matrix(circuit):
        n = len(circuit)
        adj = [[0] * n for _ in range(n)]
        for i in range(n):
            if circuit[i]:
                for j in circuit[i]:
                    adj[i][j] = 1
        return adj

    def is_d_regular(adj, d):
        for row in adj:
            if sum(row) != d:
                return False
        return True

    def rank_of_root_system(adj):
        n = len(adj)
        I = [[int(i == j) for j in range(n)] for i in range(n)]
        A = [row[:] + [-1] for row in adj]
        A += [I[i] + [0] * (n - 1) for i in range(n)]
```

```
        m = len(A)
        n += 1
        rank = 0
        for j in range(n):
            pivot_row = next((i for i in range(rank, m) if A[i][j]), None)
            if pivot_row is not None:
                A[pivot_row], A[rank] = A[rank], A[pivot_row]
                for i in range(m):
                    if i != rank:
                        factor = A[i][j] / A[rank][j]
                        A[i][j:] = [A[i][k] - factor * A[rank][k] for k in range(j, n + 1)]
                rank += 1
        return rank

    def dimension_of_lie_algebra(adj):
        return rank_of_root_system(adj)

    n = random.randint(5, 30)
    max_depth = random.randint(5, 40)
    circuit = generate_random_circuit(n, max_depth)
    adj = adjacency_matrix(circuit)

    if not is_d_regular(adj, n - 1):
        return {
            "metric_name": "Rank of Root System",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "Mapping undefined for non-d-regular graph"
        }

    rank = rank_of_root_system(adj)
    dim_lie_algebra = dimension_of_lie_algebra(adj)

    return {
        "metric_name": "Rank of Root System",
        "metric_value": rank,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": rank <= max_depth and dim_lie_algebra <= n,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```python
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d42878c3.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d42878c3.py", line 69, in run_trial
    adj = adjacency_matrix(circuit)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d42878c3.py", line 35, in adjacency_mat
    adj[i][j] = 1
    ~~~~~~^^^
TypeError: list indices must be integers or slices, not list
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with appropriate error handling to ensure it completes and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 15653 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 10136 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 10282 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 9129 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 40666 |

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14599 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 9423 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 12006 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 27015 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 148908 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/fa2dcbe02a24.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fa2dcbe02a24.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/fa2dcbe02a24.tar.gz` (if generated)